ECE 593 Exclusion List
APB System Verification - Team 2
Spring 2026 (V1.0)

The list includes exclusions made in our code coverage analysis. Some of these exclusions are a result of the limitations of the RTL with fixed values, unrealistic scenarios with respect to the APB protocol, and the limited address space usage by our design.

Note: these exclusions are based on the assumption of running 1000 tests (direct + random).

APB_Master

Line #	Type	Condition	Reasoning
162	Condition	(((!o_psel)) & o_penable & i_pready & ((~o_pwrite)))	A peripheral must be selected to drive the valid signal
162	Condition	(((!o_psel)) & o_penable & i_pready & ((~o_pwrite)))	penable is necessary for enabling the peripheral to have it respond on pready. Not a valid transaction
N/A	Toggle	i_addr_in[31:9]	The address space of the system only takes up a small section of the address bus. Also, words are 4-byte aligned, so indexes [1:0] are never toggled.
N/A	Toggle	o_addr_in[31:9]	The address space of the system only takes up a small section of the address bus. Also, words are 4-byte aligned, so indexes [1:0] are never toggled.

APB_Slave

Line #	Type	Condition	Reasoning

55, 58-59, 72, 83	Line	54 `    if ((i_pwrite)&&(WAIT_WRITE=='0')) begin`
		`//Write commnad and no wait states`
		55 `    o_pready<=1'b1;`
		56 `    end`
		57 `    else if ((~i_pwrite)&&(WAIT_READ=='0')) begin`
		`//Read commnad and no wait states`
		58 `      o_pready<=1'b1;`
		59 `      o_prdata<=mem[i_paddr[REG_NUM+WORD_LEN-1:WORD_LEN]];`
		`//Updating the o_prdata together with the rising edge of pready`
		60 `    end`
		61 `    else o_pready<=1'b0;`
		62 `  end`
		63
		64 `  else if ((i_pwrite)&&(i_psel))`
		`Write commnad with 'WAIT_WRITE' wait states`
		65 `    if (count_pready==$bits(count_pready)'(WAIT_WRITE-1)) begin`
		66 `      o_pready<=1'b1;`
		67 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		68 `    end`
		69 `    else if (count_pready==$bits(count_pready)'(WAIT_WRITE))`
		70 `      o_pready<=1'b0;`
		71 `    else`
		72 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		73
		74 `  else if ((~i_pwrite)&&(i_psel))`
		`//Read command with 'WAIT_READ' wait states`
		75 `    if (count_pready==$bits(count_pready)'(WAIT_READ-1)) begin`
		76 `      o_pready<=1'b1;`
		77 `      o_prdata<=mem[i_paddr[REG_NUM+WORD_LEN-1:WORD_LEN]];`
		`//Updating the o_prdata with the rising edge of pready`
		78 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		79 `    end`
		80 `    else if (count_pready==$bits(count_pready)'(WAIT_READ))`
		81 `      o_pready<=1'b0;`
		82 `    else`
		83 `      count_pready<=count_pready+$bits(count_pready)'(1);`
			The affected lines are tied to the set waitstates, which are pre-configured parameters. Thus, configuring the design with waitstates will not exercise these parts

55, 58, 72, 83	Branch	54 ⇨ `    if ((i_pwrite)&&(WAIT_WRITE=='0')) begin`
		`//Write commnad and no wait states`
		55 `    o_pready<=1'b1;`
		56 `    end`
		57 `    else if ((~i_pwrite)&&(WAIT_READ=='0')) begin`
		`//Read commnad and no wait states`
		58 `      o_pready<=1'b1;`
		59 `      o_prdata<=mem[i_paddr[REG_NUM+WORD_LEN-1:WORD_LEN]];`
		`//Updating the o_prdata together with the rising edge of pready`
		60 `    end`
		61 `    else o_pready<=1'b0;`
		62 `  end`
		63
		64 `  else if ((i_pwrite)&&(i_psel))`
		`//Write commnad with 'WAIT_WRITE' wait states`
		65 `    if (count_pready==$bits(count_pready)'(WAIT_WRITE-1)) begin`
		66 `      o_pready<=1'b1;`
		67 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		68 `    end`
		69 `    else if (count_pready==$bits(count_pready)'(WAIT_WRITE))`
		70 `      o_pready<=1'b0;`
		71 `    else`
		72 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		73
		74 `  else if ((~i_pwrite)&&(i_psel))`
		`//Read command with 'WAIT_READ' wait states`
		75 `    if (count_pready==$bits(count_pready)'(WAIT_READ-1)) begin`
		`==> MISSING_ELSE`
		76 `      o_pready<=1'b1;`
		77 `      o_prdata<=mem[i_paddr[REG_NUM+WORD_LEN-1:WORD_LEN]];`
		`//Updating the o_prdata with the rising edge of pready`
		78 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		79 `    end`
		80 `    else if (count_pready==$bits(count_pready)'(WAIT_READ))`
		81 `      o_pready<=1'b0;`
		82 `    else`
		83 `      count_pready<=count_pready+$bits(count_pready)'(1);`
		84
			Same reason as exclusion for line, waitstates are predefined, so these branches can not be taken

74	Condition	`else if ((~i_pwrite)&&(i_psel))`	Due to the if branch that precedes this which is the write branch, this condition can not evaluate to (0, 1). pwrite = 1 will evaluate first in the write if branch.
69, 80	Condition	`else if (count_pready==$bits(count_pready)'(WAIT_WRITE))` `else if (count_pready==$bits(count_pready)'(WAIT_READ))`	This condition will be hardcoded to always evaluate to 1 since WAIT_READ/WRITE are fixed parameters
N/A	Toggle	i_p_addr[31:9]	The address space of the system only takes up a small section of the address bus. Also, words are 4-byte aligned, so indexes [1:0] are never toggled.
N/A	Toggle	o_pslverr	No error logic for memory devices
N/A	Toggle	count_pready[2:0]	Since our waitstates are configured to small values, not all bits of count_pready are toggled

APB_Timer

Line #	Type	Condition	Reasoning
75	Branch (1.1)	if (i_pwrite && WAIT_WRITE == 0)	Parameter driven; the timer is parameterized with WAIT_WRITE > 0. This condition mathematically evaluates to FALSE, making the true branch unreachable in this test condition.
79	Branch (1.2, 1.3)& Condition	((timer_idx < NUM_TIMERS) ? t_reg[timer_idx] : '0)	Parameter drive; ternary logic is located inside the WAIT_READ == 0 block. Because WAIT_READ > 0, the parent block is locked out.
91	Condition	(count_pready == 3'(WAIT_WRITE))	Unreachable state. APB protocol dictates the Master drops penable the cycle after pready goes high (at WAIT_WRITE - 1). Dropping penable resets the timer's counter, making it physically impossible to reach the value.
95	Branch (1.7)	else if (count_pready == 3'(WAIT_WRITE))	Unreachable state. Counter resets when the Master terminates the access phase, this branch cannot execute.
105	Condition	(count_pready == 3'(WAIT_READ))	Unreachable state. Master terminates the read access phase before the wait-state counter reaches this value.
109	Branch (1.11)	else if (count_pready == 3'(WAIT_READ))	Unreachable state. Not possible to execute branch under APB bus traffic.
115	Condition	(o_pready && i_psel && timer_addr_error)(Exclude T, F, - combination)	Unreachable state: Verdi expects a state where o_pready = 1 while i_psel = 0. However, the RTL strictly defaults o_pready to 0 unless i_psel is active.
96	Line	count_pready <= count_pready + ... (1);	Parameter driven: Wait state counter increment logic. Only executes if WAIT_WRITE >=

			2. Unreachable under current parameterization (WAIT <= 1).
110	Line	count_pready <= count_pready + ... (1);	Parameter driven: Wait state counter increment logic. Only executes if WAIT_READ >= 2. Unreachable under current parameterization (WAIT <= 1).

apb_bus_if

Line #	Type	Condition	Reasoning
N/A	Toggle	paddr[31:9]	The address space of the system only takes up a small section of the address bus. Also, words are 4-byte aligned, so indexes [1:0] are never toggled.

apb_external_if

Line #	Type	Condition	Reasoning
N/A	Toggle	paddr[31:9]	The address space of the system only takes up a small section of the address bus. Also, words are 4-byte aligned, so indexes [1:0] are never toggled.